

A TUTORIAL ON GENERATIVE AI AND SIMULATION MODELING INTEGRATION

Mohammad Dehghani, Sahil Belsare, and Negar Sadeghi

Department of Mechanical and Industrial Engineering
Northeastern University
360 Huntington Ave, Boston, MA 02115, USA

ABSTRACT

This tutorial paper explores the applicability of Generative AI (GenAI), particularly Large Language Models, within the context of simulation modeling. The discussion is organized around three key perspectives: *Generation*, *Execution*, and *Analysis*. Building on the 2025 edition of this tutorial, the present revision contributes a fully reproducible code repository, an in-browser live demonstration, an updated agentic-AI workflow built on the Model Context Protocol (MCP), and an empirical comparison of frontier LLMs on the same simulation-generation prompt. The paper offers conceptual guidance and hands-on examples to support researchers and practitioners integrating GenAI into simulation environments.

1 INTRODUCTION

Some of the most revolutionary products appear to be conceived in a single night, yet they are almost always built on decades of quiet, accumulating research. The simulation modelling community knows this story well. What looks today like a mature, multi-method discipline is the product of more than half a century of incremental progress, from the first simulation languages of the 1960s (GPSS, SIMSCRIPT, SLAM), through the GUI revolution of the 1990s (ProModel, Arena), to the multi-method platforms of the 2000s such as AnyLogic, FlexSim, and Simio (Figure 1).

Artificial intelligence has followed a strikingly parallel arc: rule-based and expert systems in the 1960s, classical machine learning in the 1990s, deep learning in the 2010s, and generative AI in the early 2020s (Figure 1). Each wave reshaped what the technology could do; none arrived overnight. The point is not that 2025 was the year of generative AI; it is that 2025 was the year the two timelines started intersecting at the level of practitioner workflows.

The 2025 edition of this tutorial (?) introduced a three-perspective framework for thinking about that intersection in the context of discrete-event simulation. The three perspectives were Generation, Execution, and Analysis. The present paper is the continuation of that tutorial. It is updated for a year in which generative AI has reshaped what is possible at a speed without precedent in computing history. ChatGPT reached one hundred million monthly users within two months of launch, the fastest adoption curve ever recorded for a consumer technology (?). Frontier models that struggled with discrete-event code in 2025 now produce working SimPy scripts on the first attempt. The Model Context Protocol (?) has emerged as a standard way for language models to invoke external tools. And reproducible runnable artefacts, rather than purely conceptual tutorials, are increasingly what the community asks for.

This paper makes the following contributions:

- A revised tutorial on integrating generative AI into the discrete-event simulation lifecycle, updated to reflect frontier-model capabilities and tooling as of 2026.
- A working agentic-AI workflow built on the Model Context Protocol that allows language models to construct simulation models by calling exposed simulation primitives, rather than generating monolithic source files.
- An empirical comparison of three frontier large language models on the same simulation-generation prompt, scored on first-try success, code quality, and key-performance-indicator agreement.

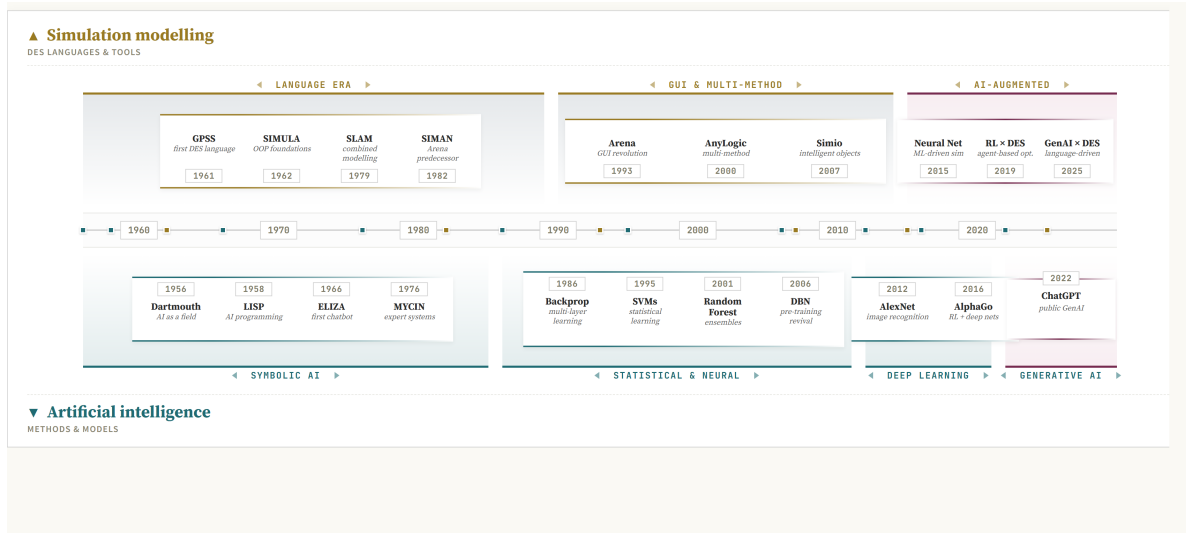


Figure 1: Parallel evolution of simulation modelling and artificial intelligence. The two timelines develop largely independently for half a century and begin to intersect in the practitioner workflow only in the 2020s.

- A fully reproducible accompaniment to the paper: a public source-code repository and an in-browser live demonstration of the running example, available at <https://github.com/mdehghani86/wsc2026-genai-simulation-tutorial> and <https://mdehghani86.github.io/wsc2026-genai-simulation-tutorial/>.

To ground these contributions in a concrete artefact, a multi-stage emergency department (ED) model is used as a single running case study throughout the paper. Patients arrive at the ED, are triaged into severity bands, may pass through imaging or laboratory services, and are then either admitted to inpatient beds or discharged. The case is intentionally rich enough to expose where current LLMs succeed and where they break. It requires branching, state tracking, multiple resource pools, and quantitative validation against analytical bounds. Section 3 defines the case in detail. Subsequent sections show how each stage of the GenAI-driven simulation workflow operates on it. The paper culminates in a side-by-side comparison of three implementation strategies in Section 8, namely vibe-coding, multi-agent generation, and MCP-driven tool use.

The rest of the paper is organised as follows. Section 2 provides a brief primer on large language models. Section 3 introduces the ED running example. Section 4 presents the three-phase GenAI-driven simulation framework. Sections 5, 6, and 7 elaborate the Creation, Execution, and Experimentation phases. Section 8 introduces the agentic-AI extension via the Model Context Protocol, Section 9 reports the empirical comparison of frontier LLMs, and Section 10 describes the reproducible artefacts. Section 11 concludes.

2 LARGE LANGUAGE MODELS IN SIXTY SECONDS

2.1 How LLMs Work

2.2 Controlling Output: Temperature and Top-k/Top-p

3 RUNNING EXAMPLE: EMERGENCY DEPARTMENT

4 GENAI-DRIVEN SIMULATION FRAMEWORK

5 CREATION

5.1 Problem Formulation (Phase 0)

Why Phase 0 is a phase. Before any code is written, the modeller has to settle what is being modelled and why. We treat this stage as Phase 0 to mark it as separate from the three downstream phases of Creation, Execution, and Experimentation. Phase 0 produces no code and no model file. It produces a written agreement on three items. The first is the problem to be solved. The second is the scope and boundaries of the system under study. The third is the set of key performance indicators (KPIs) against which the simulation will be judged. In a traditional study these three items are spread across requirements meetings, white-board sketches, and the opening of a project brief. A recent Harvard analysis of professional GenAI use found ideation to be one of its most common roles (?). What we add here is structure. The conversation is the same, but the output is constrained to a fixed schema so that the artefact can be carried forward unchanged. All prompts shown in this paper were issued to Anthropic’s Claude¹ and reproduced verbatim from the chat transcript.

Prompt and response. Figure 2 shows the Phase 0 prompt and response for the ED running example. The prompt is doing three things at once. It states the request in natural language. It supplies a short note that anchors the model in the specific facility and study question. And it asks the LLM to consult recent literature so that the returned KPIs reflect community practice rather than the modeller’s prior habits. The response is constrained to a fixed three-row schema. Forcing the structure has two practical effects. First, the output is directly comparable across projects and across language models. Second, it draws attention to whichever cell is weakest, which is almost always *Scope and boundaries*, and lets that conversation happen up front rather than after a two-hundred-line script has already been written against an unstated assumption. The caveat that Sun et al. raised about LLM ideation in general applies here (?). The model is recombining published practice, not inventing a new study design. The structured schema is what turns that recombination into a usable artefact.

Carrying Phase 0 forward. The three rows of the response are not a deliverable in isolation. They are the inputs to the next three phases. *Scope and boundaries* determines which resources and processes appear in the model file generated in Phase 1b (Section 5.3). If the row excludes ambulance routing, no SimPy resource is created for it in Phase 1 and no narration mentions it in Phase 2. The *problem definition* in the first row is the line that the Execution-phase narration (Section 6) must ultimately answer. A trace that does not address it has run, but has not produced evidence. The *KPIs* in the third row become the success criteria of Phase 3 (Section 7). Every reported number, every dashboard chart, and every scenario comparison is judged against this row. We revisit this wiring at the start of each downstream section, so the reader has a single answer to “where did this number come from?”

5.2 Input Modelling (Phase 1a)

Why the test is the authority. Phase 1a takes the *Scope and boundaries* row from Phase 0 and turns it into the random-variate generators that Phase 1b will need. The 2025 edition of this tutorial framed input modelling as one of several activities the LLM could help with, and the WSC 2026 literature has already

¹Specifically, claude-opus-4-7. Section 9 reports an empirical comparison against two other frontier models on the same simulation-generation prompt.

PROMPT	
I am planning a discrete-event simulation of an emergency department. Help me formulate the study as a structured problem statement.	
NOTES small community hospital, two triage tiers, the team wants to evaluate staffing and routing rule changes.	
SEARCH recent literature (2023–2025) on emergency-department simulation studies and pull common KPIs.	
RETURN a single table with three rows: <i>Problem definition</i> , <i>Scope & boundaries</i> , and <i>KPIs</i> . Keep each cell to two or three short sentences.	
RESPONSE	
ELEMENT	STATEMENT
1 Problem definition	Reduce length of stay in the emergency department and identify whether triage-rule changes or staffing changes deliver the larger improvement under fluctuating arrival rates.
2 Scope & boundaries	From walk-in arrival to admit-or-discharge. Two triage tiers (urgent, non-urgent), a shared imaging and laboratory pool, and fixed bed pools. Ambulance routing, no-shows, and inpatient bed flow are out of scope.
3 KPIs	Mean and 95th-percentile length of stay; door-to-doctor time per triage tier; bed and imaging utilisation; daily throughput.

Figure 2: Phase 0 of the GenAI-driven simulation workflow for the ED running example. The prompt fixes the response schema to three rows (problem definition, scope and boundaries, KPIs) so that the artefact is directly reusable in the downstream phases. The literature-search instruction grounds the KPI choice in published ED simulation studies rather than the modeller’s prior habits.

pushed further in that direction. SimLLM (?) fine-tunes code LLMs specifically on SimPy queueing data, and the LLM-driven DES framework of ? delegates input fitting to a multi-agent pipeline. Both are powerful, but both also keep the language model as the final arbiter of the distribution choice. We take a deliberately narrower position. Goodness-of-fit is a solved statistical problem. Distribution-free tests have been understood since ? and packaged in `scipy.stats` for two decades (?). An LLM that picks the distribution by inspection is throwing away that work and adding a non-reproducible step in its place. Phase 1a therefore uses the LLM only at the two ends of the pipeline, namely proposing candidates and writing the verdict, and gives the test the final word in between.

The prompt-test-interpret pipeline. The companion application built for this paper² runs Phase 1a entirely in the browser via Pyodide. The user picks a demo trace or uploads a CSV. The application validates the file by checking for a numeric column or timestamps, a sample size of at least 30, no negative values, and any zeros worth flagging. Timestamps are then converted to inter-arrival times. The LLM is prompted with a short numerical summary of the data, namely sample size, mean, standard deviation, coefficient of variation, and skewness. It is asked for two or three candidate continuous distributions, one rationale each, and *no fit*. The application then fits each candidate with `scipy.stats`, runs Kolmogorov–Smirnov, Anderson–Darling, and a binned chi-square test against the fitted CDF, and ranks the candidates by KS p-value. Figure 3 shows the result on a deliberately bursty arrival trace where all three of the LLM’s candidates are rejected at the 5% level. The empirical histogram and the LLM proposals look reasonable in isolation, but the Q-Q plot and the test column make the failure visible. The data are non-stationary (rush hours mixed with off-peak), and no single marginal distribution can absorb that. The verdict bar in the figure is what the LLM writes after reading the test output. The recommendation it produces, refit per hour-of-day, is what the modeller would have reached on their own with classical input-modelling practice. The point is not that the LLM is wrong, but that the test is what made it visible.

Hand-off to Phase 1b. The artefact produced by Phase 1a is a single Python expression. It is a callable that returns one inter-arrival sample. The companion application emits this expression directly as a copy-pasteable SimPy line, with a banner noting whether the test accepted the fit or whether the closest-rejected candidate was used as a fallback. Phase 1b consumes that line as the arrival generator in the model file generated in Section 5.3. If the banner reports a rejection, the Phase 1b prompt is instructed

²<https://mdehghani86.github.io/wsc2026-genai-simulation-tutorial/>, tab *Input modelling*. Source at <https://github.com/mdehghani86/wsc2026-genai-simulation-tutorial>.

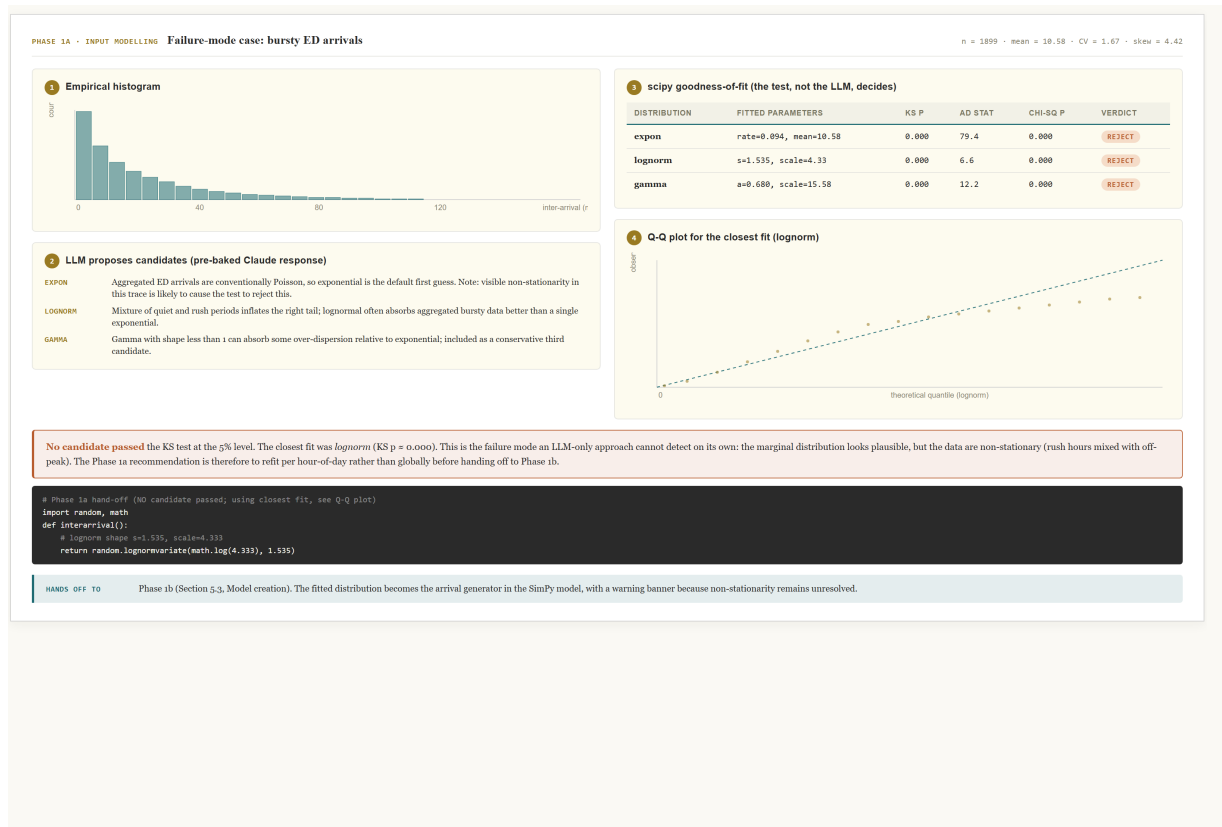


Figure 3: Phase 1a on a deliberately bursty ED arrival trace. The LLM proposes three candidate distributions with one-line rationales; `scipy.stats` fits each and runs KS, AD, and chi-square goodness-of-fit. All three candidates are rejected at the 5% level. The Q-Q plot shows systematic deviation in the upper tail, and the verdict bar (written by the LLM after reading the test output) recommends refitting per hour-of-day. The same screen on a clean exponential trace produces an accepted fit and a copy-pasteable SimPy line; both demos are available in the companion application.

to refuse the global fit and to ask for hour-of-day stratification instead. The connectivity is therefore not narrative. The row of the Phase 0 table that asked for “arrival rate” is answered by a Python line that the next phase reads, and the goodness-of-fit verdict travels with it.

5.3 Model Generation (Phase 1b): Vibe Coding with a Validation Gate

What template editors already do. Off-the-shelf simulation packages have offered template-based modelling for two decades. Tools such as Simio, Arena, FlexSim, and AnyLogic let the modeller drag a server, a queue, or a routing decision onto a canvas, configure parameters in dialogs, and publish a runnable model without writing code. The template editor enforces invariants by construction. A server has a capacity. A queue feeds exactly one server. A routing decision lists every outgoing arc. The parameter dialog refuses out-of-domain values. Phase 1b asks whether the same invariants can be enforced when the model is generated as Python code rather than placed on a canvas. The answer in this paper is yes, but only if the invariants are made an explicit, automated check rather than left for the modeller’s eye.

The five-part prompt. The 2025 edition of this tutorial introduced a five-part structured prompt for SimPy generation, comprising Role, Goal, Scenario, Inputs, and Output. We retain the structure but feed it from the Phase 0 schema and the Phase 1a hand-off. The Role asks the LLM to write as a simulation engineer. The Goal is to produce a runnable SimPy file for the Phase 0 problem. The Scenario is the

row of *Scope and boundaries* from Phase 0, copied verbatim, so the LLM cannot drift into adjacent ED variants the modeller did not ask for. The Inputs section pastes the arrival generator from Phase 1a (a single Python expression) together with the fitted distributions for service times. The Output section asks for one Python file with no dependencies beyond SimPy and the standard library. SimLLM (?) has shown that fine-tuned 7B-class models can generate executable SimPy on a similar prompt template, and ? reach the same conclusion via a multi-agent pipeline. Both confirm that frontier-model zero-shot prompting is a defensible default for a tutorial reader, which is the route we take here.

The validation gate. Frontier LLMs are powerful but probabilistic. On the same prompt, two runs can produce structurally different SimPy files, and silent bugs in the generated code can survive a casual read. We therefore put a fixed five-check validation gate between the generated file and the downstream phases (Figure 4). Process logic checks that the patient flow in the generated file matches the Phase 0 scope row end to end. Connectivity checks that each producer wires into the declared consumer with no orphan resources. Parallel-versus-sequential checks that a pool with capacity c actually serves c patients in parallel. This is one of the silent failure modes that motivated the WSC 2025 lessons-learned review of this case study. A single per-pool “manager” process collapses effective parallelism to one even when `simpy.Resource(capacity=c)` is declared correctly. Distribution checks verify that sampled service times match the Phase 1a fit in mean, variance, and support. Theory checks compare realised utilisation, mean queue length, and mean waiting time against closed-form Erlang-C and Allen–Cunneen predictions for the matching $M/G/c$ system. If any of the five checks fails, the failing check is appended to the next prompt verbatim and the LLM is asked to regenerate only the affected section. If three rounds of regeneration do not pass, the case is handed off to the multi-agent or MCP path of Section 8. The artefact that leaves Phase 1b is therefore not just a SimPy file. It is a SimPy file plus a passing validation report. Either both travel forward to Phase 2, or neither does.

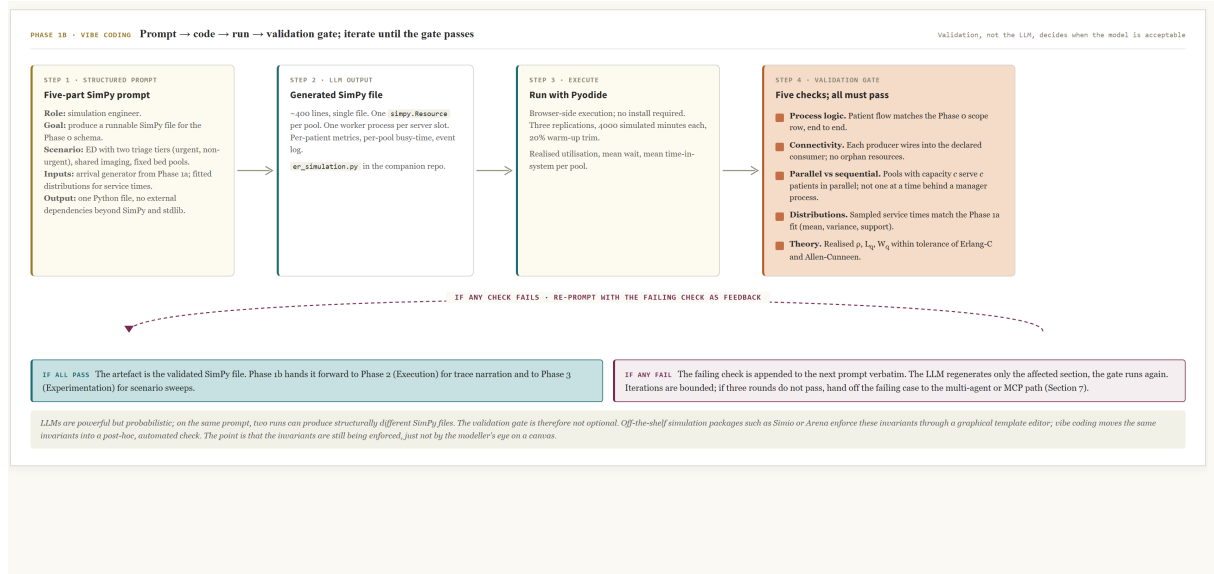


Figure 4: Phase 1b vibe-coding loop. The structured prompt produces a SimPy file. Pyodide runs the file. A five-check validation gate (process logic, connectivity, parallelism, distributions, theory) decides whether the artefact may travel forward. If any check fails, the failing check is appended to the next prompt and iterations are bounded. The point is not that LLMs cannot write SimPy, but that on probabilistic generators the invariants must be enforced after the fact rather than by construction. Off-the-shelf template editors such as Simio or Arena enforce the same invariants at draw time. The gate is the code-based equivalent.

The Phase 1b artefact, the validated SimPy file together with its passing report, is published as `phaselb/er_simulation.py` in the companion repository (Section 10). Phase 2 reads that file as input and produces the trace narration (Section 6). Phase 3 runs scenario sweeps against it and scores them against the Phase 0 KPI list (Section 7).

5.4 Retrieval-Augmented Generation (RAG)

6 EXECUTION

6.1 Simulation Narration

6.2 Process Orchestration

6.3 Outcome Forecasting

7 EXPERIMENTATION

7.1 Result Interpretation

7.2 Scenario Generation

8 AGENTIC AI VIA MODEL CONTEXT PROTOCOL

9 EMPIRICAL COMPARISON OF FRONTIER LLMS

10 REPRODUCIBILITY AND LIVE DEMONSTRATION

11 CONCLUSION

ACKNOWLEDGMENTS

AUTHOR BIOGRAPHIES

MOHAMMAD DEHGHANIMOHAMMADABADI is an Associate Teaching Professor in the Department of Mechanical and Industrial Engineering at Northeastern University. His research focuses on developing intelligent simulation frameworks integrated with optimization and AI-based models to support decision-making, with applications in healthcare, manufacturing, and supply chain. His email address is m.dehghani@northeastern.edu.

SAHIL BELSARE is a simulation scientist with expertise in digital twin technologies and simulation modeling. He applies machine learning models to enhance decision-making in complex systems, focusing on optimization and reinforcement learning integrated with simulation platforms. His email is belsare.s@northeastern.edu.

NEGAR SADEGHI is a Ph.D. student in Industrial Engineering at Northeastern University. Her research focuses on developing intelligent simulation models for pharmaceutical supply chains and production-distribution systems. Her email address is sadeghi.ne@northeastern.edu.